UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ARQUITECTURA DE COMPUTADORES Y ENSAMBLADORES

MANUAL TÉCNICO:

NOMBRE	CARNET
Christopher Miguel Angel Ramos Ascencio	202200057
Esmeralda Del Rosario Guillen Veliz	201901002
Sophía Raquel Pérez Marroquin	202405047
Nelson Andres Santa Cruz Gil	202202728

Requerimientos del sistema

Categoría	Requerimiento	Detalle
Sistema Físico	Raspberry Pi 3/4 o PC equivalente	Con sistema operativo basado en Linux (Raspberry Pi OS o Ubuntu recomendado)
Lenguaje	Python	Versión 3.8 o superior
Dependencias Python	Librerías necesarias	RPi.GPIO o gpiozero, paho-mqtt, pymongo, Adafruit_DHT, adafruit-circuitpython-tcs34725
Servidor Backend	Node.js	Versión 16 o superior con Express
Base de Datos	MongoDB	Versión 5.0 o superior
Comunicación	Broker MQTT	Instalado y configurado
Frontend	React Node.js	Versión 18 o superior para visualización

BACKEND (python)

Se implementa el protocolo MQTT para la comunicación entre los sensores y el backend, utilizando un modelo de tipo publish/subscribe, adecuado para sistemas IoT por su eficiencia y bajo consumo de recursos. En este proyecto, los sensores publican datos (como temperatura y humedad) en un broker MQTT, mientras que el backend actúa como suscriptor, encargado de recibir, procesar y almacenar la información en una base de datos MongoDB.

El flujo de comunicación es el siguiente:

1. Los sensores envían datos (temperatura, humedad, etc).
2. Estos datos se publican en el broker MQTT.
3. El backend se suscribe a los tópicos correspondientes.
4. Al recibir un mensaje, el backend lo procesa y lo almacena en la base de datos.

Configuración MQTT:
El backend se conecta a un broker local a través de la dirección *mqtt://localhost:1883*, que corresponde al host y puerto estándar del servicio MQTT. Cuando ya se estableció la conexión, el sistema se suscribe a la jerarquía de tópicos *nave/sensores/#,* lo que permite recibir todos los mensajes publicados por los sensores. Cada mensaje recibido contiene el tópico, que identifica el tipo de sensor, y el valor enviado, el cual es transformado a un formato adecuado antes de ser almacenado en la base de datos.

backend > config > mqtt.js > client.on("message") callback > data > value

```javascript
const mqtt = require("mqtt")
const SensorReading = require("../models/sensorReading")

const client = mqtt.connect("mqtt://localhost:1883")

client.on("connect", () => {
    console.log("Conectado a MQTT")

    client.subscribe("nave/sensores/#")
})

client.on("message", async (topic, message) => {
    console.log("Mensaje recibido:",topic, message.toString())

    const data = new SensorReading({
        sensor: topic,
        value: parseFloat(message.toString())
    })

    await data.save()
})

module.exports = client
```

Configuración del servidor:
El backend del sistema está desarrollado utilizando *Express.js*, el cual permite la gestión de solicitudes y comunicación con los diferentes módulos del sistema. El servidor integra la configuración del cliente MQTT, permitiendo que se suscriba a los tópicos de los sensores, y pueda recibir información en tiempo real.
Se definen distintas rutas para organizar la lógica de la aplicación, como */sensors, /commands* y */stats*. Las cuales permiten gestionar la consulta, control y análisis de datos.

```
backend > JS server.js > ...
const express = require("express")
const cors = require("cors")
const connectMongo = require("./config/mongo")

require("./config/mqtt")

const sensorsRoutes = require("./routes/sensors")
const commandsRoutes = require("./routes/commands")
const statsRoutes = require("./routes/stats")

const app = express()

app.use(cors())
app.use(express.json())

connectMongo()

app.use("/sensors", sensorsRoutes)
app.use("/commands", commandsRoutes)
app.use("/stats", statsRoutes)

app.get("/", (req,res)=>{
    res.send("Backend nave activo")
})

app.listen(3000, ()=>{
    console.log("Servidor corriendo en puerto 3000")
```

Sensores
Esta carpeta tiene todas las funcionalidades de los sensores utilizando la librería "**RPi.GPIO**"

Configuración: Inicial:

```
class Environment:

    # ---------------- CONFIG ----------------
    DHT_SENSOR = Adafruit_DHT.DHT11
    DHT_PIN = 1
    SOIL_PIN = 1

    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883

    TOPIC_TEMP = "nave/sensores/temperatura"
    TOPIC_AMBIENTE = "nave/sensores/ambiente"
    TOPIC_SOIL = "nave/sensores/humedad"
    TOPIC_ALERT = "nave/alertas/criticas"
```

Configuracion de la Clase: Fire_detector.py

```python
class FireDetector:

    # ---------------- CONFIG ----------------
    MQ_PIN = 14
    LED = 4
    FAN1 = 0
    FAN2 = 0

    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883

    TOPIC_SENSOR = "nave/sensores/gas"
    TOPIC_ALERT = "nave/alertas/criticas"

    THRESHOLD = 1  # HIGH = peligro
```

Configuracion de la Clase: gate.py

```python
class Gate:

    # ---------------- CONFIG ----------------
    SERVO = 15
    BUTTON_OPEN = 10
    BUTTON_CLOSE = 9

    OPEN_ANGLE = 90
    CLOSED_ANGLE = 0

    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883
    TOPIC_COMMAND = "nave/actuadores/compuertas"

    DEBOUNCE_TIME = 0.3
```

Configuracion de la Clase: LCDdisplay.py

```python
class Display:

    # ---------------- MQTT ----------------
    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883
    TOPIC_MSG = "nave/control/mensajes"

    def __init__(self, systems):

        self.systems = systems

        # ---------------- LCD ----------------
        self.lcd = LCD(0x27, 1, 16, 2, True)
        self.lcd.clear()
        self.lcd.backlight(True)
```

Configuracion de la Clase: meteor_detector.py

```python
class MeteorDetector:

    # ---------------- CONFIG ----------------
    TRIG_PIN = 23
    ECHO_PIN = 24
    BUZZER = 0
    Y_LED = 27

    SPEED_OF_SOUND_CM_S = 34300
    TIMEOUT_S = 0.02
    SAMPLES = 5

    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883
    TOPIC_SENSOR = "nave/sensores/proximidad"
    TOPIC_ALERT = "nave/alertas/criticas"
```

Configuracion de la Clase: turret.py

```python
class Turret:

    # ---------------- CONFIG ----------------
    PINS = [12, 7, 8, 25]

    BUTTON_LEFT = 6
    BUTTON_RIGHT = 5
    BUTTON_FIRE = 13

    LASER = 4
    BUZZER = 0

    MQTT_BROKER = "localhost"
    MQTT_PORT = 1883
    TOPIC_COMMAND = "nave/actuadores/torreta"

    DEBOUNCE = 0.2
```

Inicializacion del sistema atraves del main.py

```python
def main():

    print(" Iniciando sistema de la nave...")

    systems = {}

    try:
        # -------- INICIALIZAR --------
        fire = FireDetector()
        meteor = MeteorDetector()
        disguise = Disguise()
        env = Environment()
        turret = Turret()
        gate = Gate()

        systems = {
            "fire": fire,
            "meteor": meteor,
            "disguise": disguise,
            "env": env,
            "gate": gate,
            "turret": turret
        }
```

Levantar el backend

cd ~/Escritorio/Arqui1-P1-main/backend
npm install
node server.js

Levantar el mqtt

sudo systemctl start mosquitto
sudo systemctl enable mosquitto
systemctl is-active mosquitto

FRONTEND (React)

El frontend fue desarrollado con React con el objetivo de construir una interfaz interactiva capaz de monitorear el estado de la nave y enviar órdenes hacia el sistema físico. Su funcionamiento se basa en una arquitectura por componentes, donde cada parte del dashboard representa información específica o una funcionalidad concreta.

El flujo general del frontend comienza cuando el usuario abre la aplicación en el navegador. En ese momento, el componente principal consulta al backend para solicitar el estado de los sensores y las estadísticas. Los datos obtenidos se almacenan en estados internos y luego se presentan de forma visual mediante barras, indicadores, paneles de estado y controles interactivos.

A lo largo del uso del sistema, el frontend actualiza periódicamente la información para reflejar cambios recientes en la telemetría. Esto permite mostrar condiciones como niveles de gas, temperatura, proximidad y otros datos relevantes del entorno monitoreado. Al mismo tiempo, el usuario puede interactuar con los controles para mover la torreta, disparar, abrir o cerrar compuertas, activar camuflaje y enviar mensajes al LCD. Cada una de estas acciones genera una solicitud al backend, que luego se encarga de reenviar la orden hacia el sistema físico mediante MQTT.

También se implementó una segunda vista dedicada a gráficas, accesible desde un botón del dashboard principal.

La razón de implementar el frontend de esta manera fue lograr una interfaz clara, modular y fácil de mantener. React permite separar la lógica visual, el manejo de estado y la comunicación con el backend, haciendo más sencillo extender el sistema.

- <u>configuración de la Clase api.js:</u>

La clase o archivo api.js se encarga de centralizar toda la comunicación entre el frontend y el backend. Su función principal es evitar que la lógica de consumo de la API quede dispersa en varios componentes de la interfaz. Dentro de este archivo se define una constante base de conexión hacia el backend, lo cual permite que todas las solicitudes se dirijan al servidor Express que corre en el puerto 3000. A partir de esa base se construyen los endpoints del sistema, por ejemplo para consultar /sensors, /stats, /commands, /messages y el histórico de sensores.

Además, api.js incorpora una función de manejo de respuestas que valida si la petición fue exitosa. Si el servidor responde correctamente, el archivo convierte la respuesta a formato JSON y la devuelve al frontend.

En resumen, api.js funciona como una capa de integración entre el frontend y el backend. Su implementación permite mantener el código ordenado, reutilizable y más fácil de mantener, además de simplificar el desarrollo de componentes visuales como el dashboard principal y la ventana de gráficas.

```jsx
const handleResponse = async (response) => {
  }

  return response.json();
};

export const getSensors = async () => {
  const response = await fetch("/sensors");
  return handleResponse(response);
};

export const getStats = async () => {
  const response = await fetch("/stats");
  return handleResponse(response);
};

export const sendCommand = async ({ topic, command, value }) => {
  const response = await fetch("/commands", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      topic,
      command,
      value,
    }),
```

- <u>Configuración Inicial en la clase app.jsx:</u>

El archivo App.jsx representa el componente principal del frontend. En este archivo se construye la interfaz del dashboard y se gestiona la mayor parte del estado visual del sistema. Aquí se inicializan variables de estado para representar sensores, actuadores, mensajes, estadísticas y animaciones gráficas de la nave.

En primer lugar, se importan los recursos necesarios: React, estilos, la imagen de la nave y las funciones auxiliares provenientes de api.js. A partir de esto, el componente define distintos estados con useState, por ejemplo para almacenar el valor actual de gas, temperatura, humedad, proximidad, ángulo de la torreta, mensaje LCD y estadísticas generales. Estos estados permiten que la interfaz cambie dinámicamente conforme llegan datos nuevos desde el backend o cuando el usuario realiza una acción.

El archivo utiliza useEffect para ejecutar procesos automáticos al cargar la aplicación. Esto permite que el dashboard se mantenga actualizado sin necesidad de recargar la página manualmente. Una vez recibidos los datos, se transforman mediante funciones auxiliares y se actualizan los estados correspondientes.

Otro aspecto importante de App.jsx es que controla las acciones del usuario. Por ejemplo, cuando se mueve el control de la torreta, cuando se presiona el botón de disparo, cuando se abren o cierran compuertas, cuando se activa el camuflaje o cuando se envía un mensaje al LCD. Cada una de estas acciones invoca funciones que realizan solicitudes al backend, permitiendo que el sistema físico responda a los controles definidos en la interfaz web.

Además, en este componente se implementa la lógica visual del dashboard. Se define la distribución en panel izquierdo, centro y derecho, el estado general de la nave, los indicadores LED, las barras de sensores y la presentación de estadísticas. La implementación de App.jsx fue importante porque permitió concentrar en un solo componente la lógica principal del tablero de control, integrando datos en tiempo real, acciones del usuario y elementos visuales del sistema. Gracias a esto, el frontend funciona como una consola de monitoreo y control centralizada.

```
12  } from "./services/api .js";
13
14  export default function App() {
15    const [gas, setGas] = useState(0);
16    const [temp, setTemp] = useState(0);
17    const [humedad, setHumedad] = useState(0);
18    const [distancia, setDistancia] = useState(0);
19    const [angulo, setAngulo] = useState(120);
20    const [mensaje, setMensaje] = useState("");
21    const [animacion, setAnimacion] = useState("");
22    const [mensajeEnviado, setMensajeEnviado] = useState(false);
23    const [alertaGeneral, setAlertaGeneral] = useState(false);
24    const [anguloAnim, setAnguloAnim] = useState(false);
25    const [cargando, setCargando] = useState(true);
26    const [error, setError] = useState("");
27    const [stats, setStats] = useState({
28      disparos: 0,
29      tiempoCamuflaje: "00:00:00",
30      alertasGas: 0,
31      alertasMeteorito: 0,
32      totalLecturas: 0,
33    });
34
```

- configuración en la clase de GraphsPage.jsx:

Prácticamente acá se implementó todo lo relacionado a las gráficas del sistema en tiempo real

```
GraphsPage.jsx U ×
frontend > src > GraphsPage.jsx > ...
import { useEffect, useMemo, useState } from "react";
import {
  LineChart,
  Line,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  BarChart,
  Bar,
  Cell,
}
from "recharts";
import { getLatestSensors, getSensorHistory } from "./services/api.js";
import "./graphs.css";

const slides = [
  { key: "temperatura", title: "Temperatura - últimas 24 horas" },
  { key: "ambiente", title: "Ambiente / humedad - últimas 24 horas" },
  { key: "gas", title: "Gas - histórico reciente" },
  { key: "proximidad", title: "Proximidad - histórico reciente" },
  { key: "color", title: "Color detectado" },
];

export default function GraphsPage() {
  const [index, setIndex] = useState(0);
  const [temperaturaData, setTemperaturaData] = useState([]);
  const [ambienteData, setAmbienteData] = useState([]);
  const [gasData, setGasData] = useState([]);
  const [proximidadData, setProximidadData] = useState([]);
  const [colorDocs, setColorDocs] = useState([]);
  const [latest, setLatest] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadGraphs = async () => {
      try {
        const [
          temperatura,
          ambiente,
          gas,
          proximidad,
          color,
```

- configuración en la Clase vite.config.js:

El archivo vite.config.js define la configuración principal del entorno de desarrollo del frontend. En este proyecto se utilizó Vite como herramienta de construcción y servidor de desarrollo, ya que permite levantar aplicaciones React de forma rápida y ligera.

Dentro de este archivo se configura el plugin de React, necesario para que Vite pueda procesar correctamente los componentes JSX del proyecto. Además, se establece la configuración del servidor de desarrollo, incluyendo el uso de un proxy para redirigir ciertas rutas del frontend hacia el backend.

La importancia de este archivo radica en que permite trabajar localmente con frontend y backend en puertos distintos. Su implementación fue necesaria para simplificar las pruebas locales, permitir hot

reload de React y facilitar la integración entre la parte visual y la parte de servicios.

```
4   export default defineConfig({
5     plugins: [react()],
6     server: {
7       proxy: {
8         "/sensors": {
9           target: "http://localhost:3000",
10          changeOrigin: true,
11        },
12        "/stats": {
13          target: "http://localhost:3000",
14          changeOrigin: true,
15        },
16        "/commands": {
17          target: "http://localhost:3000",
18          changeOrigin: true,
19        },
20        "/messages": {
21          target: "http://localhost:3000",
22          changeOrigin: true,
23        },
24        "/events": {
25          target: "http://localhost:3000",
26          changeOrigin: true,
27        },
28      },
29    },
30  });
```

Levantar el frontend

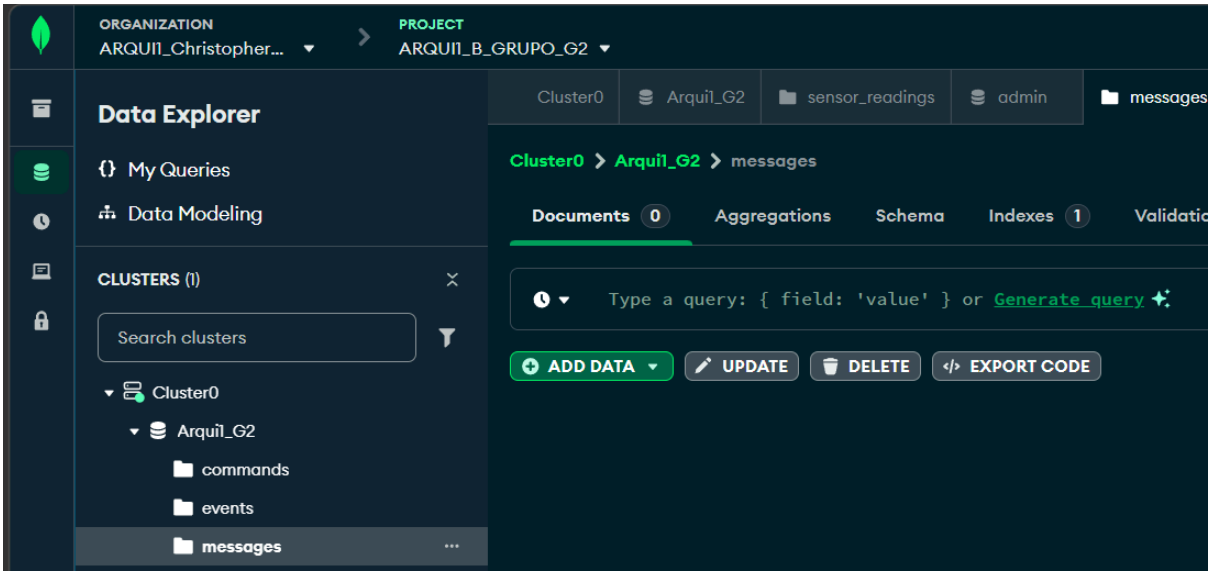
IMPORTANTE —-- En otra terminal:

cd ~/Escritorio/Arqui1-P1-main/frontend
npm install
npm run dev

Mongo DB ATLAS

Para la estructura de base de datos no relacional o “colecciones” se utilizo la herramienta web “MongoDB_ATLAS”
Dentro de esta herramienta se ha creado la organización: ARQUI1_ChristopherRamos
En el cual se encuentra el proyecto: ARQUI1_B_GRUPO2

Creación del Cluster
El proyecto contiene un Cluster0 con la base de datos Arqui1_G2



USUARIOS:
Se crearon dos usuarios dentro del cluster:

usuario1: dbUser
Contraseña: dbUserPassword

usuario2: dbRaspberry
Contraseña: dbRaspberryPassword

Como instalar las dependencias de mongoDB

Instalacion de la librería para python:
python -m pip install "pymongo[srv]==3.13"

NOTA: VERIFICA TU VERSION DE PYTHON:
cmd | python –version | cambiar segun la version que tengas

```
C:\Users\kellt>python --version
Python 3.13.12

C:\Users\kellt>
```

URL para conectarse al cluster0:
mongodb+srv://dbUser:<db_password>@cluster0.aghdbrn.mongodb.net/?appName=Cluster0

RUTA utilizada en el archivo .env en la carpeta de mongoDB

```
MONGODB_URI=mongodb+srv://dbUser:dbUserPassword@cluster0.aghdbrn.mongodb.net/Arqui1_G2?retryWrites=true&w=majority&appName=Cluster0

MONGODB_DB=Arqui1_G2
```

Clases utilizadas

1. mongodb_manager.py

- Conecta a MongoDB Atlas
- Proporciona métodos para guardar datos

2. mqtt_to_mongodb.py

- Escucha MQTT broker (está suscrito a todos los tópicos)
- Cuando recibe datos de main.py, llama a mongodb_manager para guardar
- Es el intermediario entre MQTT y MongoDB

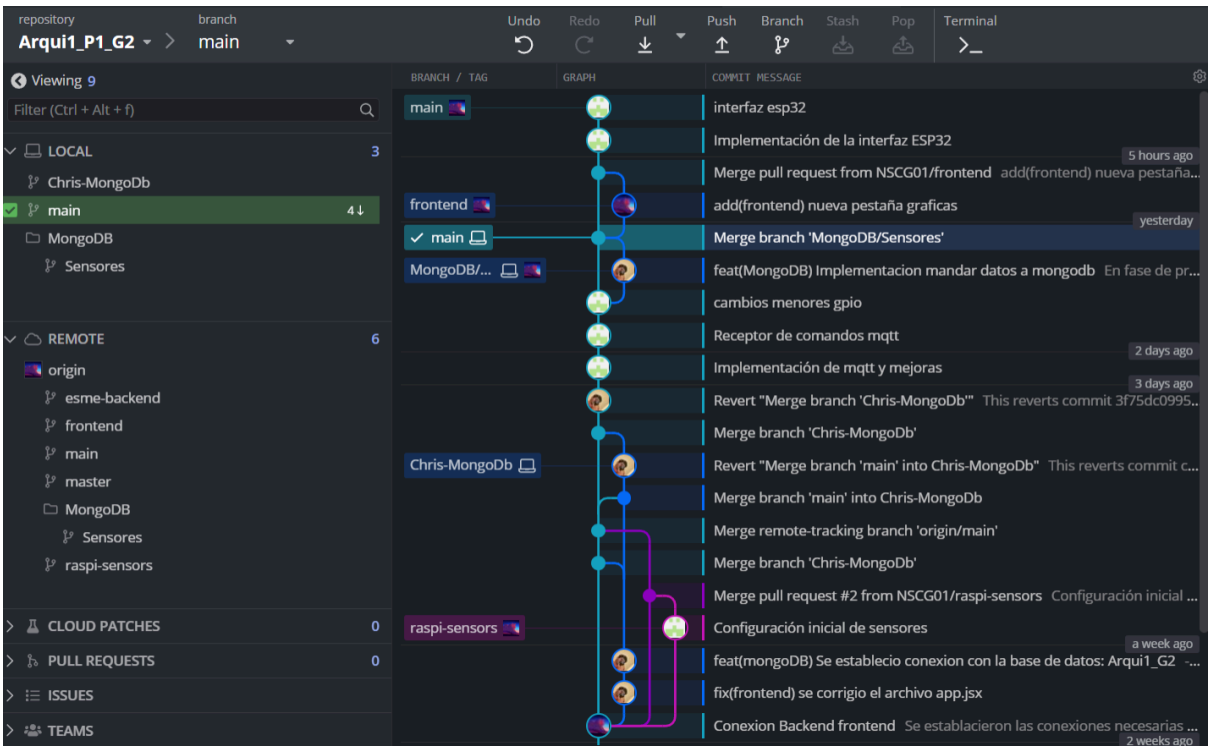
3. check_mongodb.py

- Consulta MongoDB y muestra cuántos documentos hay
- Muestra el último documento guardado

Link:
Cluster0 Data | Cloud: MongoDB Cloud

Link de grabaciones del funcionamiento
https://drive.google.com/drive/folders/1R741MCkuF50hULByrAZBA1HAbyvuOY6U?usp=sharing

Anexos



Flujo de trabajo en git Kraken